UNIVERSIDAD RAFAEL LANDÍVAR

Facultad de Ingeniería

Ingeniería en Informática y Sistemas — Programación Web

Catedrático: Ing. Ronald Leonel Ayala Garrido

Ciclo: Segundo Semestre 2026

ENTREGABLE FINAL: APLICACIÓN WEB FUNCIONAL

Proyecto: ReserveHub — Sistema Web de Reservas de Espacios Universitarios

---

1. INTEGRANTES DEL EQUIPO DE TRABAJO

Nombre Completo	No. Carné	Rol
Walter José Herrera Salvatierra	1109722	Estudiante
Boris Garzona	1218122	Estudiante
Jonathan García	1045222	Estudiante
Samuel Ruiz	1228722	Estudiante
Giancarlo Ortega	1230222	Estudiante
Mario López	2477123	Estudiante

2. ENLACES OFICIALES DEL DESPLIEGUE EN PRODUCCIÓN

Componente	Plataforma	Enlace Oficial
Frontend Web	Vercel	https://proyecto-final-progra-web-theta.vercel.app
Backend REST API	Render	https://reservehub-backend-881f.onrender.com/api/health
Base de Datos	Render PostgreSQL	Instancia gestionada con Prisma ORM (en la nube)

Componente	Plataforma	Enlace Oficial
Repositorio GitHub	GitHub	https://github.com/JonyGarciaaa/ProyectoFinalPrograWeb
Pipeline CI/CD	GitHub Actions	https://github.com/JonyGarciaaa/ProyectoFinalPrograWeb/actions

3. CREDENCIALES DE ACCESO PARA EVALUACIÓN

Para la revisión y pruebas del catedrático, se establecieron cuentas preconfiguradas que cubren los roles contemplados en el sistema:

Rol	Correo Electrónico	Contraseña	Alcance de Permisos
Administrador	`admin@reservehub.com`	`Admin123!`	Acceso a /admin. Registro, modificación y deshabilitación de espacios; gestión de estados de reserva (confirmar, cancelar, finalizar) y consulta del listado de usuarios.
Usuario General	`jonathan@reservehub.com`	`User123!`	Consulta de disponibilidad por fecha y franjas horarias; creación de reservaciones y cancelación de reservas propias en /my-reservations.
Usuario Secundario	`walter@reservehub.com`	`User123!`	Validación de concurrencia y prevención de duplicidad de horarios entre usuarios distintos.

4. RESUMEN DE ARQUITECTURA Y TECNOLOGÍAS

El proyecto fue desarrollado utilizando una arquitectura desacoplada de tres niveles:

- **Frontend:** React 19 estructurado con TypeScript y Vite. Para los estilos se implementó Tailwind CSS bajo diseño responsivo sin uso de plantillas prediseñadas. La gestión del estado de autenticación se maneja con React Context API y almacenamiento del token JWT.
- **Backend:** API REST desarrollada con Node.js (versión 22) y Express 5 en TypeScript. El código está organizado bajo una estructura de capas: Rutas, Controladores, Servicios, Middlewares y Validadores. Las validaciones de datos de entrada se procesan mediante la librería Zod.
- **Persistencia de Datos:** PostgreSQL versión 16 como sistema gestor de base de datos relacional y Prisma ORM para el acceso tipado a datos, migraciones y ejecución del sembrado inicial de datos (seed).
- **Seguridad:** Cifrado de contraseñas mediante función hash bcryptjs con factor de coste de 10 rondas de salting; autenticación y autorización mediante tokens JWT en el encabezado Authorization; cabeceras HTTP configuradas con Helmet; y control de acceso basado en roles (RBAC).
- **Pruebas Automatizadas:** Suites de pruebas unitarias y de integración para Backend (Vitest y Supertest) y Frontend (@testing-library/react) integradas en el pipeline de GitHub Actions.
- **Contenerización:** Configuración multi-etapa con Docker y Docker Compose para la ejecución local de PostgreSQL, Backend y Frontend con Nginx.

5. REGLAS DE NEGOCIO VALIDADAS

- **RN-01:** Un espacio no puede registrar dos reservaciones con traslape en la misma fecha y rango de horas. Condición lógica: (nuevaInicio < existenteFin) Y (nuevaFin > existenteInicio).
- **RN-02:** Se requiere inicio de sesión y token JWT válido para crear una reservación.
- **RN-03:** Un usuario únicamente tiene permisos para cancelar sus propias reservaciones activas.
- **RN-04:** El administrador tiene acceso a consultar y actualizar el estado de todas las reservaciones.

- **RN-05:** No se pueden realizar reservaciones sobre espacios marcados como deshabilitados.
- **RN-06:** La hora de inicio de una reservación debe ser estrictamente menor a la hora de finalización.
- **RN-07:** No se permite la creación de reservaciones para fechas pasadas respecto a la fecha actual del sistema.
- **RN-08:** Cada reservación queda vinculada formalmente a un usuario y a un espacio existente.
- **RN-09:** Los estados permitidos para una reserva son: PENDING, CONFIRMED, CANCELLED y COMPLETED.

6. EJECUCIÓN LOCAL CON DOCKER (EVALUACIÓN EN AULA)

Para la revisión local del proyecto mediante contenedores, se requiere tener Docker Desktop en ejecución y utilizar el siguiente comando desde la raíz del repositorio:

```
docker compose up -d --build
```

Direcciones locales generadas:

- Frontend Web: `http://localhost:5173`
- Backend API: `http://localhost:4000/api/health`
- PostgreSQL: puerto `5432`

7. MATRIZ DE CUMPLIMIENTO DE LA RÚBRICA DE CALIFICACIÓN (25 PUNTOS)

Criterio Evaluado	Puntaje	Estado
1. Análisis y diseño	3.0 pts	Completado
2. Frontend — React + TypeScript	5.5 pts	Completado
3. Backend — Express + TypeScript	4.0 pts	Completado
4. Base de datos — PostgreSQL + Prisma	3.0 pts	Completado

Criterio Evaluado	Puntaje	Estado
5. Seguridad	2.0 pts	Completado
6. Pruebas de software	1.5 pts	Completado
7. Git/GitHub y trabajo colaborativo	1.5 pts	Completado
8. Docker + CI/CD	1.5 pts	Completado
9. Despliegue, accesibilidad y rendimiento	1.5 pts	Completado
10. Documentación y defensa técnica	1.5 pts	Completado
TOTAL	**25.0 / 25.0 pts**	**100% Finalizado**
